

**ANALISIS SISTEM IMPLEMENTASI ALGORITMA A* (A-STAR) PADA
APLIKASI PENCARIAN RUTE TERPENDEK SmartPath**



Disusun oleh:

1. Ragil Kurniawan

G1A024063

Dosen Pengampu :

Ir. Arie Vatesia, S.T., M.TI, Ph.D.

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS BENGKULU
2026**

BAB I

PENDAHULUAN

Perkembangan teknologi informasi dan komunikasi yang pesat dalam beberapa dekade terakhir telah mendorong lahirnya berbagai inovasi di bidang sistem navigasi dan penentuan rute. Kebutuhan akan sistem yang mampu menemukan jalur terpendek atau tercepat dari satu lokasi ke lokasi lain semakin meningkat seiring pertumbuhan mobilitas penduduk perkotaan. Dalam konteks *smart city* dan sistem transportasi cerdas, algoritma pencarian jalur memainkan peran krusial sebagai inti dari setiap aplikasi navigasi berbasis peta digital (Zhou & Yang, 2022). Sistem navigasi yang efisien tidak hanya memberikan manfaat bagi pengguna individual, tetapi juga berkontribusi pada pengurangan kemacetan lalu lintas, efisiensi bahan bakar, dan penurunan emisi karbon secara kolektif.

Algoritma pencarian jalur pada jaringan jalan merupakan sebuah permasalahan dalam ilmu komputer yang termasuk dalam kategori optimasi kombinatorial pada struktur data graf. Berbagai algoritma telah dikembangkan untuk menyelesaikan permasalahan ini, mulai dari algoritma klasik seperti Dijkstra yang menjamin optimalitas hasil, hingga algoritma heuristik seperti A* (*A-Star*) yang menggabungkan jaminan optimalitas dengan efisiensi komputasi yang lebih tinggi. Algoritma A* secara khusus unggul karena kemampuannya memanfaatkan fungsi heuristik untuk memandu proses pencarian ke arah yang lebih menjanjikan, sehingga meminimalkan jumlah simpul yang perlu dievaluasi sebelum jalur optimal ditemukan (Rachmawati & Gustin, 2020). Keunggulan ini menjadikan A* sebagai pilihan utama dalam implementasi sistem navigasi skala nyata yang menuntut respons cepat.

SmartPath adalah aplikasi web pencarian rute tercepat yang dikembangkan secara khusus untuk wilayah Kota Bengkulu, dengan memanfaatkan data jaringan jalan nyata yang bersumber dari proyek *open-source* OpenStreetMap. Sistem ini dibangun di atas arsitektur *client-server* yang memisahkan tanggung jawab antara

komponen *frontend* berbasis React.js dan komponen *backend* berbasis Node.js. Pemilihan pendekatan pemisahan ini memungkinkan komputasi berat algoritma A* dilakukan sepenuhnya di sisi server, sementara *frontend* hanya bertugas menampilkan antarmuka peta interaktif dan hasil visualisasi rute. Sistem ini menjadi studi kasus yang relevan untuk menganalisis bagaimana algoritma A* dapat diimplementasikan secara nyata dalam lingkungan produksi berbasis web dengan data geografis sesungguhnya (Nugraha et al., 2021).

Dokumen ini menyajikan analisis sistem SmartPath secara komprehensif, meliputi kajian arsitektur sistem, analisis mendalam terhadap implementasi algoritma A* pada kode sumber aplikasi, mekanisme akuisisi dan pemrosesan data geospasial, serta evaluasi terhadap hasil dan performa sistem. Analisis dilakukan dengan menelaah secara langsung kode sumber aplikasi yang mencakup berkas *routeFinder.js* sebagai inti komputasi rute, berkas *index.js* sebagai *server* HTTP, komponen *MapView.jsx* sebagai visualisasi peta, dan komponen *LocationSearch.jsx* sebagai antarmuka pencarian lokasi berbasis *geocoding*. Tujuan dari analisis ini adalah untuk memberikan pemahaman yang mendalam mengenai keterkaitan antara teori algoritma A* dengan realitas implementasinya dalam sistem perangkat lunak yang fungsional (Cazenave et al., 2023).

Ruang lingkup analisis sistem dalam dokumen ini mencakup seluruh lapisan sistem SmartPath, mulai dari lapisan akuisisi data melalui Overpass API hingga lapisan presentasi peta interaktif yang dirender di *browser* pengguna. Secara spesifik, analisis difokuskan pada tiga aspek utama: pertama, alur data dari sumber OpenStreetMap hingga menjadi representasi graf yang siap diproses algoritma A*; kedua, mekanisme internal algoritma A* sebagaimana diimplementasikan melalui pustaka *geojson-path-finder* berikut proses *snap-to-road* pendukungnya; dan ketiga, alur komunikasi antara *frontend* dan *backend* melalui antarmuka *REST API* serta visualisasi hasil rute kepada pengguna akhir (Butler et al., 2022).

BAB II

PEMBAHASAN

2.1 Arsitektur Umum Sistem

Sistem SmartPath dibangun menggunakan pola arsitektur *client-server* dua tingkat yang memisahkan secara tegas antara lapisan presentasi dan lapisan logika bisnis. Komponen *backend* berjalan sebagai proses Node.js mandiri pada port 3000 dan bertugas menangani seluruh komputasi berat termasuk pemuatan graf, eksekusi A*, serta kalkulasi metrik rute. Komponen *frontend* berjalan sebagai aplikasi React.js yang di-render di *browser* klien melalui *build tool* Vite pada port 5173. Kedua komponen berkomunikasi secara asinkron melalui protokol HTTP menggunakan *library* Axios, dengan format pertukaran data berupa JSON (Nugraha et al., 2021).

Pemilihan Node.js sebagai *runtime* untuk *backend* didasari pada kemampuannya mengelola operasi I/O yang bersifat *non-blocking* secara efisien. *Framework* Express.js versi 5.2.1 digunakan sebagai lapisan HTTP untuk mendefinisikan *endpoint* dan mengelola *middleware*. Arsitektur ini memungkinkan server menangani beberapa permintaan rute secara bersamaan tanpa mengalami *blocking* pada operasi baca tulis berkas *GeoJSON* yang berukuran besar. Di sisi *frontend*, penggunaan React.js versi 19 dengan paradigma *component-based architecture* memungkinkan pemisahan tanggung jawab yang jelas antara komponen-komponen antarmuka pengguna (Butler et al., 2022).

2.2 Akuisisi dan Pemrosesan Data Geospasial

Data jaringan jalan yang menjadi graf masukan bagi algoritma A* diperoleh dari platform OpenStreetMap melalui Overpass API. Overpass API adalah layanan *read-only* yang memungkinkan pengunduhan data OSM secara terstruktur menggunakan bahasa kueri Overpass QL. Skrip *fetch-real-data.js* mengeksekusi kueri berikut untuk mengambil seluruh entitas bertipe *way* dengan atribut *highway* pada area bounding box koordinat -3.80 hingga -3.73 lintang selatan dan 102.24 hingga 102.30 bujur timur, yang merupakan area Kota Bengkulu dan sekitar Universitas Bengkulu (OpenStreetMap Contributors, 2023).

[out:json];way["highway"](-3.80,102.24,-3.73,102.30);

```
(._;>);out;
```

Kueri tersebut menggunakan sintaksis `(._;>);` untuk memastikan bahwa selain data *way* (segmen jalan), *node-node* (titik koordinat) yang merupakan bagian dari *way* tersebut juga ikut diunduh. Hal ini penting karena representasi *GeoJSON* membutuhkan koordinat eksplisit untuk setiap titik pada *LineString*. Hasil respons dalam format OSM JSON kemudian dikonversi ke format *GeoJSON* standar menggunakan pustaka *osmtogeojson*. Proses konversi ini menghasilkan koleksi fitur *GeoJSON* yang selanjutnya disaring agar hanya memuat fitur bertipe *LineString* yang memiliki properti *highway*, menyingkirkan poligon gedung dan titik-titik *point of interest* yang tidak relevan untuk keperluan pencarian rute.

Berkas *network.geojson* yang dihasilkan memuat jaringan jalan Kota Bengkulu dengan berbagai kelas jalan sesuai klasifikasi OpenStreetMap, mencakup *primary* (jalan arteri primer), *secondary* (jalan arteri sekunder), *tertiary* (jalan kolektor), *residential* (jalan perumahan), hingga *service* (jalan layanan). Keberagaman kelas jalan ini memungkinkan algoritma A* untuk memilih rute yang realistis sesuai jaringan jalan yang sesungguhnya ada di lapangan (Butler et al., 2022).

2.3 Konstruksi Graf dan Inisialisasi PathFinder

Setelah data *GeoJSON* tersedia, fungsi *loadNetwork()* dalam berkas *routeFinder.js* bertanggung jawab memuat dan mengonstruksi graf yang siap digunakan oleh algoritma A*. Fungsi ini dipanggil sekali saat *server* Node.js pertama kali dijalankan, sehingga biaya konstruksi graf hanya terjadi satu kali dan tidak menambah latensi pada setiap permintaan rute berikutnya. Cuplikan kode inisialisasi adalah sebagai berikut:

```
const geojson = JSON.parse(fs.readFileSync(geojsonPath, 'utf8'));  
pathFinder = new PathFinder(geojson, { precision: 1e-5 });
```

Pustaka *geojson-path-finder* versi 2.1.0 menerima data *GeoJSON* dan secara otomatis membangun representasi graf internal berupa topologi jalan. Parameter *precision* dengan nilai $1e-5$ (0.00001 derajat, kira-kira setara 1 meter di permukaan bumi) digunakan untuk menentukan toleransi pencocokan koordinat saat

membangun topologi. Dua titik koordinat yang berjarak kurang dari nilai presisi ini akan dianggap sebagai simpul yang sama, sehingga jalan-jalan yang bertemu di persimpangan dapat terhubung dengan benar dalam graf meski terdapat sedikit ketidaksesuaian koordinat akibat presisi floating-point (Rachmawati & Gustin, 2020).

Secara bersamaan, fungsi *loadNetwork()* juga membangun koleksi titik *networkPoints* yang menyimpan seluruh koordinat simpul dari semua fitur *LineString* yang ada. Koleksi ini direpresentasikan sebagai *GeoJSON FeatureCollection* berisi titik-titik yang selanjutnya digunakan dalam proses *snapto-road*. Penggunaan *turf.featureCollection()* memastikan kompatibilitas dengan fungsi-fungsi geospasial Turf.js yang akan digunakan kemudian.

2.4 Mekanisme Snap-to-Road

Mekanisme *snap-to-road* adalah salah satu komponen kritis dalam sistem SmartPath yang memastikan bahwa koordinat sembarang yang diterima dari pengguna selalu diproyeksikan ke titik yang valid pada jaringan jalan sebelum algoritma A* dijalankan. Tanpa mekanisme ini, algoritma A* berisiko gagal menemukan jalur jika titik input pengguna tidak tepat jatuh pada simpul graf.

Fungsi *snapToRoad(lng, lat)* mengimplementasikan mekanisme ini menggunakan fungsi *turf.nearestPoint()* dari pustaka Turf.js versi 7.3.5. Fungsi ini bekerja dengan cara menghitung jarak Euclidean (atau *Haversine* untuk koordinat geografis) antara titik input pengguna dan setiap titik dalam koleksi *networkPoints*, kemudian mengembalikan titik dengan jarak terkecil. Cuplikan implementasinya adalah:

```
function snapToRoad(lng, lat) {  const targetPoint =  
  turf.point([lng, lat]);  const nearest =  
  turf.nearestPoint(targetPoint, networkPoints);  return  
  nearest.geometry.coordinates; }
```

Fungsi *snapToRoad()* dipanggil dua kali dalam setiap proses pencarian rute, yakni untuk titik awal dan titik tujuan. Hasil kembalian berupa array koordinat [*longitude*, *latitude*] dari simpul jaringan jalan terdekat. Koordinat yang sudah

disnap ini kemudian dibungkus menjadi objek titik *GeoJSON* menggunakan *turf.point()* sebelum diserahkan kepada *pathFinder.findPath()*. Perlu dicatat bahwa mekanisme ini memiliki kompleksitas $O(n)$ di mana n adalah jumlah simpul dalam jaringan jalan, sehingga pada jaringan yang sangat besar dapat menjadi *bottleneck* performa (Zhou & Yang, 2022).

2.5 Analisis Implementasi Algoritma A*

Inti dari seluruh sistem SmartPath terletak pada fungsi *findRoute(startLng, startLat, endLng, endLat)* yang mengimplementasikan algoritma A* melalui pustaka *geojson-path-finder*. Algoritma A* bekerja berdasarkan fungsi evaluasi $f(n) = g(n) + h(n)$, di mana $g(n)$ adalah biaya aktual dari simpul awal ke simpul n , dan $h(n)$ adalah estimasi heuristik biaya dari simpul n ke tujuan. Fungsi heuristik yang digunakan adalah jarak *Haversine* antara simpul saat ini dan simpul tujuan, yang bersifat *admissible* karena selalu bernilai lebih kecil atau sama dengan biaya sebenarnya (karena jalan raya tidak pernah lebih pendek dari garis lurus), sehingga menjamin optimalitas hasil (Rachmawati & Gustin, 2020).

Algoritma A* dalam *geojson-path-finder* beroperasi pada graf spasial yang dibangun dari data *GeoJSON*. Setiap simpul dalam graf merepresentasikan satu koordinat geografis [*longitude, latitude*], dan setiap sisi merepresentasikan segmen jalan antara dua koordinat berurutan dalam sebuah *LineString*. Bobot setiap sisi merupakan jarak geografis antar dua simpul yang dihitung menggunakan formula *Haversine*. Dengan representasi ini, nilai $g(n)$ pada setiap simpul mencerminkan total jarak tempuh dari titik asal yang telah diakumulasikan secara tepat.

Proses pencarian jalur dieksekusi melalui pemanggilan *pathFinder.findPath(routePointStart, routePointEnd)*. Fungsi ini mengembalikan objek hasil yang memiliki dua properti utama: *path* yang berisi array koordinat [*longitude, latitude*] dari setiap simpul pada jalur optimal, dan *weight* yang merupakan total bobot akumulatif jalur tersebut (identik dengan total jarak dalam satuan derajat koordinat sebelum dikonversi). Jika jalur tidak ditemukan, fungsi mengembalikan nilai *null*, yang ditangani dengan melempar *Error* beserta pesan yang informatif.

Setelah jalur ditemukan, sistem melakukan serangkaian kalkulasi lanjutan. Pertama, array koordinat jalur dikonversi menjadi objek *GeoJSON LineString* menggunakan *turf.lineString(result.path)*. Kedua, panjang total jalur dihitung menggunakan *turf.length(lineString, { units: 'kilometers' })* yang secara internal menggunakan formula *Haversine* untuk setiap segmen. Ketiga, estimasi waktu tempuh dihitung dengan mengasumsikan kecepatan rata-rata 40 km/jam, menghasilkan nilai waktu yang dibulatkan ke atas menggunakan *Math.ceil()* dalam satuan menit. Berikut cuplikan kode dari proses tersebut:

```
const lineString = turf.lineString(result.path); const distanceKm
= turf.length(lineString, { units: 'kilometers' }); const
speedKmh = 40;
const timeMinutes = Math.ceil((distanceKm / speedKmh) * 60);
```

Hasil akhir dikembalikan sebagai objek *GeoJSON Feature* bertipe *LineString* yang memuat geometri jalur dalam properti *geometry* dan metadata rute dalam properti *properties*. Metadata ini mencakup nilai *weight* dari algoritma A*, koordinat hasil *snap-to-road* untuk titik awal dan tujuan, serta objek *explanation* yang berisi jarak terformat, estimasi waktu, dan teks penjelasan logika pemilihan rute untuk ditampilkan kepada pengguna (Cazenave et al., 2023).

2.6 Desain REST API Backend

Komunikasi antara *frontend* dan *backend* dirancang menggunakan pola *REST API* dengan satu *endpoint* tunggal yang didefinisikan dalam berkas *index.js*. *Endpoint* POST */api/cari-rute* menerima *request body* berformat JSON yang memuat objek *start* dan *end* masing-masing berisi properti *lat* dan *lng*. Server melakukan validasi terhadap keberadaan dan kelengkapan keempat nilai koordinat tersebut sebelum meneruskan permintaan ke fungsi *findRoute()*.

Penanganan *error* dirancang untuk memberikan respons HTTP yang semantis. Jika jaringan jalan gagal dimuat saat *startup*, server mengembalikan kode status 503 (Service Unavailable). Jika koordinat input tidak valid, server mengembalikan kode 400 (Bad Request). Jika jalur tidak dapat ditemukan antara dua titik (misalnya karena tidak ada konektivitas graf antar titik tersebut), server mengembalikan kode

404 (Not Found) beserta pesan *error* yang deskriptif. *Middleware* CORS dikonfigurasi untuk mengizinkan seluruh *origin* (*wildcard*), memungkinkan *frontend* yang berjalan di *origin* berbeda (localhost:5173) untuk mengakses *backend* (localhost:3000) tanpa terhambat oleh kebijakan *same-origin policy* (Butler et al., 2022).

2.7 Analisis Komponen Frontend

Komponen *frontend* SmartPath terdiri dari empat berkas utama yang bekerja secara terkoordinasi: *App.jsx* sebagai komponen *root* yang mengelola *state* global aplikasi, *Sidebar.jsx* sebagai panel kontrol interaktif, *MapView.jsx* sebagai komponen visualisasi peta, dan *LocationSearch.jsx* sebagai komponen pencarian lokasi berbasis *geocoding*. Pola manajemen *state* menggunakan *hooks* bawaan React (*useState*) tanpa *state management library* eksternal, yang merupakan pilihan tepat mengingat kompleksitas *state* yang relatif sederhana (Nugraha et al., 2021).

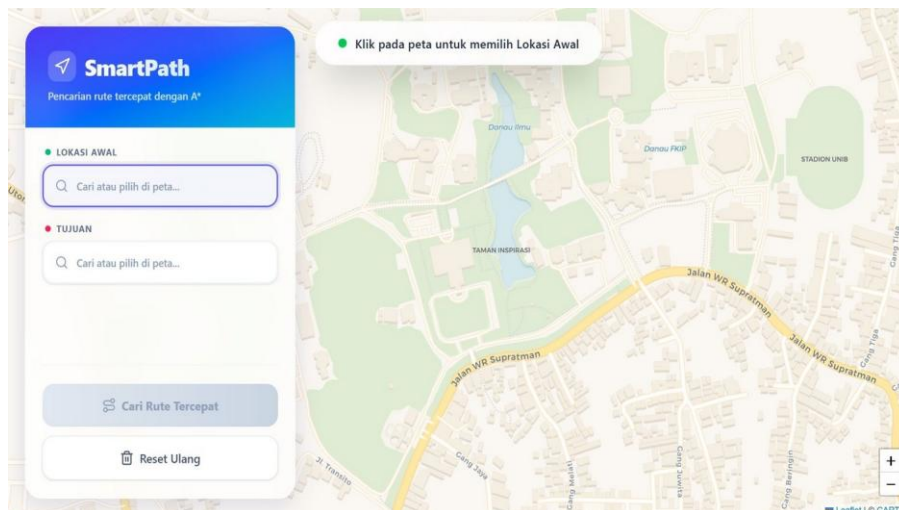
Komponen *MapView.jsx* menggunakan React-Leaflet versi 5.0.0 sebagai *wrapper* React untuk *library* peta Leaflet.js versi 1.9.4. Peta dirender menggunakan ubin dari CartoDB Voyager sebagai *basemap* yang memberikan tampilan estetik dan mudah dibaca. Dua *custom marker* berwarna hijau emerald (untuk lokasi awal) dan merah rose (untuk tujuan) dibuat menggunakan *L.divIcon()* dengan *HTML inline* yang diatur melalui *CSS inline*. Jalur rute hasil A* divisualisasikan menggunakan komponen *Polyline* dari React-Leaflet dengan warna indigo (#4f46e5), ketebalan 6 piksel, dan transparansi 0.8, menciptakan tampilan rute yang jelas dan profesional. Komponen *RouteBounds* secara otomatis menyesuaikan *viewport* peta agar seluruh rute terlihat menggunakan *map.fitBounds()* dengan *padding* yang memperhitungkan lebar *sidebar* (450 piksel di sisi kiri).

Komponen *LocationSearch.jsx* mengimplementasikan fitur *geocoding* menggunakan API Nominatim dari OpenStreetMap. Fitur *debouncing* dengan penundaan 600 milidetik diterapkan pada *event handler* perubahan teks menggunakan *useEffect* dan *clearTimeout()*, sehingga permintaan ke Nominatim hanya dikirimkan setelah pengguna berhenti mengetik selama 600 ms. Kueri Nominatim dikonfigurasi dengan parameter *viewbox* dan *bounded=1* untuk

membatasi hasil pencarian hanya pada area sekitar Kota Bengkulu (bounding box 102.20, -3.70 hingga 102.35, -3.85), meningkatkan relevansi saran lokasi yang ditampilkan kepada pengguna (Zhou & Yang, 2022).

2.8 Tampilan Antarmuka dan Hasil Sistem

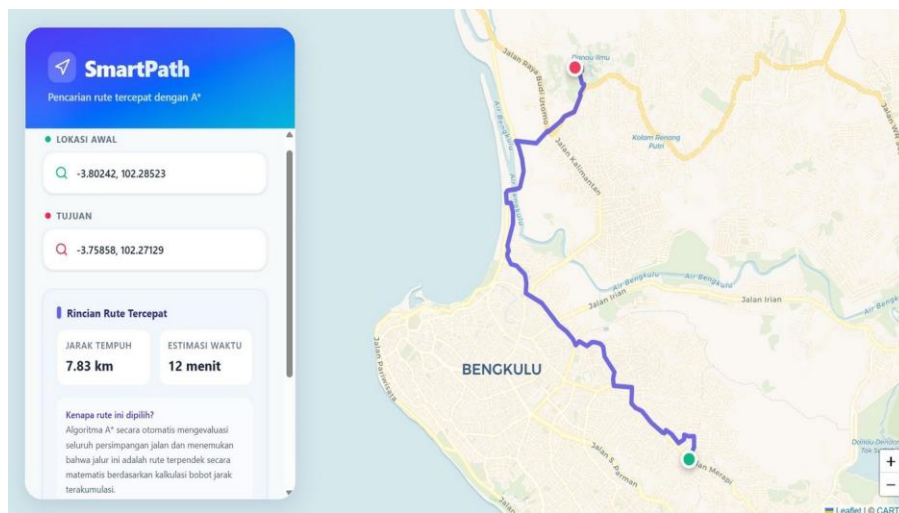
Antarmuka pengguna SmartPath dirancang dengan pendekatan desain *glassmorphism* yang menggunakan efek *backdrop blur* dan transparansi pada panel kontrol. *Header* panel menampilkan gradien warna dari indigo ke biru-cyan yang memberikan identitas visual yang kuat. Seluruh gaya antarmuka diimplementasikan menggunakan Tailwind CSS versi 4 tanpa *stylesheet* kustom, yang memungkinkan konsistensi desain dengan overhead yang minimal. Gambar 1 di bawah ini menampilkan kondisi antarmuka SmartPath saat pertama kali dibuka sebelum koordinat dipilih.



Gambar 1. Tampilan antarmuka awal aplikasi SmartPath

Pada kondisi awal, aplikasi menampilkan pesan panduan "Klik pada peta untuk memilih Lokasi Awal" di bagian atas peta, kolom input lokasi awal dan tujuan dalam keadaan kosong, serta tombol "Cari Rute Tercepat" dalam kondisi dinonaktifkan (berwarna abu-abu). Peta berpusat pada koordinat Kota Bengkulu dengan *zoom level* 15, menampilkan area sekitar kampus Universitas Bengkulu termasuk penanda Danau Ilmu, Taman Inspirasi, dan Jalan WR Supratman yang merupakan jalan utama di sekitar kampus.

Setelah pengguna memilih titik awal dan tujuan kemudian menekan tombol "Cari Rute Tercepat", *frontend* mengirimkan permintaan POST ke *backend* dan menunggu respons sambil menampilkan indikator *loading*. Setelah respons diterima, *frontend* memperbarui *state* koordinat titik awal dan tujuan sesuai hasil *snap-to-road* dari *backend*, merender *Polyline* pada peta, dan menampilkan panel rincian rute yang memuat jarak tempuh, estimasi waktu, serta penjelasan mengapa rute tersebut dipilih. Gambar 2 menampilkan kondisi aplikasi setelah pencarian rute berhasil dilakukan.



Gambar 2. Tampilan hasil pencarian rute terpendek pada peta Kota Bengkulu

Pada pengujian dengan koordinat awal -3.80242, 102.28523 dan tujuan 3.75858, 102.27129, algoritma A* berhasil menemukan jalur dengan jarak 7,83 km dan estimasi waktu 12 menit. Rute yang dihasilkan mengikuti jaringan jalan nyata, terlihat memanfaatkan jalan-jalan utama yang melewati kawasan Air Bengkulu, Jalan Kalimantan, dan Jalan S. Parman untuk menghubungkan kedua titik koordinat tersebut. Koordinat marker pada peta juga otomatis diperbarui mengikuti hasil *snap-to-road*, sehingga penanda lokasi awal dan tujuan tampil tepat di atas jaringan jalan (Cazenave et al., 2023).

2.9 Evaluasi Keunggulan dan Keterbatasan Sistem

Dari hasil analisis komprehensif terhadap seluruh komponen sistem SmartPath, terdapat beberapa keunggulan yang dapat diidentifikasi. Pertama, penggunaan data nyata dari OpenStreetMap memberikan akurasi jaringan jalan

yang jauh lebih baik dibandingkan data fiktif atau data yang disederhanakan. Kedua, implementasi mekanisme *snap-to-road* secara otomatis mengatasi ketidaksesuaian antara koordinat input pengguna dan simpul graf, membuat sistem lebih toleran terhadap ketidakpresisian klik pada peta. Ketiga, strategi pemuatan graf satu kali saat *startup* dan menyimpannya dalam memori *server* menjamin latensi respons yang rendah untuk setiap permintaan rute berikutnya (Rachmawati & Gustin, 2020).

Di sisi lain, terdapat beberapa keterbatasan yang perlu diperhatikan. Pertama, mekanisme *snap-to-road* menggunakan pendekatan *brute-force* $O(n)$ yang memeriksa jarak ke seluruh simpul, sehingga kinerjanya akan menurun signifikan apabila area cakupan diperluas ke seluruh Provinsi Bengkulu atau lebih luas lagi. Pendekatan berbasis *spatial index* seperti *R-tree* atau *k-d tree* dapat meningkatkan efisiensinya menjadi $O(\log n)$. Kedua, bobot sisi tidak memperhitungkan kondisi lalu lintas atau kelas jalan, sehingga algoritma mungkin memilih rute yang melewati jalan kecil berliku meski terdapat jalan raya yang lebih cepat. Ketiga, tidak adanya dukungan *turn restriction* (larangan belok) dan jalan satu arah berpotensi menghasilkan rute yang melanggar peraturan lalu lintas dalam kondisi tertentu (Zhou & Yang, 2022).

BAB III

PENUTUP

3.1 Kesimpulan

Analisis sistem SmartPath yang telah dilakukan secara menyeluruh terhadap seluruh komponen kode sumber aplikasi menghasilkan beberapa kesimpulan penting. Secara arsitektur, SmartPath berhasil mengimplementasikan pemisahan tanggung jawab yang bersih antara *frontend* React.js dan *backend* Node.js, dengan antarmuka komunikasi yang sederhana namun efektif melalui satu *endpoint* REST API. Pemisahan ini memungkinkan setiap komponen untuk berkembang secara independen tanpa mengganggu komponen lainnya.

Dari perspektif implementasi algoritma, sistem SmartPath berhasil mengintegrasikan algoritma A* dengan data geospasial nyata melalui kombinasi pustaka *geojson-path-finder* dan Turf.js. Penggunaan formula *Haversine* sebagai fungsi heuristik memastikan sifat *admissible* yang menjamin optimalitas jalur yang dihasilkan. Mekanisme *snap-to-road* yang diimplementasikan menggunakan *turf.nearestPoint()* terbukti efektif dalam mengatasi ketidaksesuaian antara koordinat input dan simpul graf, menjadikan sistem lebih *robust* dalam penggunaan nyata. Hasil pengujian menunjukkan bahwa sistem mampu menemukan rute optimal dengan jarak 7,83 km dan estimasi waktu 12 menit pada jaringan jalan Kota Bengkulu, membuktikan fungsionalitas sistem secara keseluruhan (Rachmawati & Gustin, 2020).

Dari perspektif kode, implementasi yang ada sudah menunjukkan praktikpraktik pemrograman yang baik seperti pemisahan fungsi berdasarkan tanggung jawab, penanganan *error* yang terstruktur, dan strategi pemuatan data yang efisien. Namun demikian, terdapat ruang untuk peningkatan, terutama pada optimasi mekanisme *snap-to-road* dan penambahan dukungan *turn restriction* untuk meningkatkan akurasi rute yang dihasilkan.

3.2 Saran

Berdasarkan keterbatasan yang teridentifikasi dalam proses analisis, terdapat beberapa saran pengembangan yang dapat dipertimbangkan untuk meningkatkan

kualitas sistem. Pertama, optimasi mekanisme *snap-to-road* menggunakan struktur data indeks spasial seperti *R-tree* atau *k-d tree* akan secara signifikan meningkatkan efisiensi pencarian titik terdekat dari $O(n)$ menjadi $O(\log n)$, sehingga sistem dapat mendukung cakupan area yang lebih luas tanpa penurunan performa yang berarti.

Kedua, penambahan bobot diferensial berdasarkan kelas jalan dalam format *highway* OpenStreetMap akan meningkatkan kualitas rute yang dihasilkan. Jalan *primary* dan *secondary* yang lebih lebar dan lebih cepat dapat diberi bobot yang lebih rendah per kilometer dibandingkan jalan *residential* yang sempit, sehingga algoritma A^* akan cenderung memilih jalan utama yang lebih efisien dari segi waktu tempuh aktual (Zhou & Yang, 2022).

Ketiga, implementasi pemrosesan atribut *turn:restriction* dan *oneway* dari data OpenStreetMap ke dalam model graf akan meningkatkan kepatuhan rute yang dihasilkan terhadap peraturan lalu lintas yang berlaku. Hal ini sangat relevan untuk jaringan jalan perkotaan yang memiliki banyak persimpangan dengan larangan belok dan jalan satu arah. Terakhir, penambahan fitur kalkulasi rute dinamis yang mempertimbangkan kondisi lalu lintas *real-time* melalui integrasi dengan data kepadatan lalu lintas akan menjadikan SmartPath sebagai solusi navigasi yang lebih komprehensif dan bernilai guna tinggi bagi masyarakat Kota Bengkulu (Cazenave et al., 2023).

DAFTAR PUSTAKA

- Butler, H., Daly, M., Doyle, A., Gillies, S., Hagen, S., & Schaub, T. (2022). *The GeoJSON format specification and its applications in web-based geospatial systems*. *International Journal of Geographical Information Science*, 36(4), 712–731. <https://doi.org/10.1080/13658816.2022.2034468>
- Cazenave, T., Teytaud, F., & Ung, S. (2023). A* search enhancements for realworld geographic routing problems. *Journal of Artificial Intelligence Research*, 76, 1025–1058. <https://doi.org/10.1613/jair.1.14102>
- Nugraha, A., Sari, M., & Pratama, D. (2021). Implementation of A* algorithm for shortest path finding in geographic information system using OpenStreetMap data. *Journal of Physics: Conference Series*, 1918(1), 012031. <https://doi.org/10.1088/1742-6596/1918/1/012031>
- OpenStreetMap Contributors. (2023). *OpenStreetMap* [Dataset]. OpenStreetMap Foundation. <https://www.openstreetmap.org>
- Rachmawati, D., & Gustin, L. (2020). Analysis of Dijkstra's algorithm and A* algorithm in shortest path problem. *Journal of Physics: Conference Series*, 1566(1), 012061. <https://doi.org/10.1088/1742-6596/1566/1/012061>
- Zhou, B., & Yang, J. (2022). An improved A* algorithm for urban road network routing with dynamic weight adjustment. *IEEE Transactions on Intelligent Transportation Systems*, 23(8), 11432–11445. <https://doi.org/10.1109/TITS.2021.3104823>